

Tolerância a Falhas

Relatório Trabalho Prático

Diogo Marques
PG55931

Francisco Ferreira
PG55942

Ivan Ribeiro
PG55950

Neste trabalho prático, pretendemos identificar e analisar possíveis vulnerabilidades do algoritmo Raft perante a falhas assertivas que comprometam as propriedades de segurança do mesmo.

Implementação do Raft

O algoritmo do Raft foi implementado em Rust.

A estrutura principal do código está dividida em módulos que separam as responsabilidades:

- `raft.rs`: Contém a lógica central do algoritmo Raft, incluindo a máquina de estados dos nodos (Seguidor, Candidato, Líder), a replicação de *logs*, e os processos de eleição.
- `maelstrom.rs`: Providencia a abstração para interagir com o ambiente *Maelstrom*, tratando da serialização/desserialização de mensagens e da comunicação entre nodos (ler do *stdin* e escrever para o *stdout*).
- `transport.rs`: Define uma interface de transporte para o Raft (`RaftTransport`) e uma implementação específica para Maelstrom (`MaelstromTransport`), desacoplando a lógica do Raft do mecanismo de transporte subjacente.
- `main.rs`: Inicializa o nodo Maelstrom, integrando o sistema com os testes `lin-kv` do Maelstrom.

Para testar a sua implementação via *Maelstrom* foi utilizado o seguinte comando:

```
java -jar maelstrom.jar test
-w lin-kv
-bin ".\target\release\pessi-raft.exe"
--time-limit 60
--node-count 3
--concurrency 10n
--rate 100
--nemesis partition
--nemesis-interval 3
```

Como resultado, conseguimos observar que o *Maelstrom* não detectou falhas no nosso algoritmo.

> Everything looks good!

No entanto, não é certo que o algoritmo esteja totalmente correto, pois o Maelstrom apenas testa um subconjunto limitado de cenários e não garante a cobertura de todos os possíveis estados ou falhas do sistema.

Verificação de estados com o [Stateright](#)

Para além dos testes de integração com o Maelstrom, recorreu-se ao [Stateright](#) para uma verificação mais formal do algoritmo Raft implementado. O [Stateright](#) é uma ferramenta de

model checking para sistemas distribuídos escritos em Rust, que permite explorar exhaustivamente os possíveis estados de um sistema e verificar se certas propriedades, a partir de código.

Funcionamento e Aplicação ao Raft

O *model checking* com [Stateright](#) envolve a definição de:

1. Um **modelo** do sistema: Este modelo descreve os estados possíveis de cada nodo Raft (e.g., `current_term`, `voted_for`, `log`, `commit_index`, `role`) e as ações que podem transitar o sistema de um estado para outro (e.g., enviar/receber uma mensagem `RequestVote`, `AppendEntries`, dar *timeout* numa eleição);
2. Um **ambiente**: Define como as ações dos nodos interagem, incluindo a rede (que pode perder, duplicar ou reordenar mensagens) e outros eventos não determinísticos (mensagens que podem ser enviadas em qualquer momento);
3. **Propriedades**: São invariantes ou condições que devem ser sempre verdadeiras em todos os estados alcançáveis do sistema.

Esta definição foi feita através de:

- implementação do Raft, como modelo;
- ambiente com uma rede que não perde, duplica ou reordena mensagens;
- uma única mensagem, que consiste em escrever o valor 42 na chave 1;¹
- uma lista de propriedades que iremos apresentar a seguir.

Propriedades Verificadas

Foram definidas e verificadas várias propriedades fundamentais do Raft para garantir a sua correção:

1. **Segurança da Eleição (Election Safety)**: No máximo um líder pode ser eleito num determinado termo.
 - Expectativa temporal: Always
2. **Coerência do Log (Log Safety)**: Se dois *logs* contêm uma entrada *committed* com o mesmo índice e termo, então os *logs* são idênticos até esse índice.
 - Expectativa temporal: Always
3. **Vivacidade do Log (Log Liveness)**: O *log* deverá progredir, isto é, haver um líder que aplique entradas no *log*. O *log* não deverá ficar vazio.
 - Expectativa temporal: Sometimes
4. **Vivacidade de Eleições (Election Liveness)**: O sistema deverá progredir de modo a que um líder seja eleito. Um líder deverá ser eleito.
 - Expectativa temporal: Sometimes

Estas propriedades, que se encontram em `src/fault/property.rs`, foram verificadas com o [Stateright](#) para o algoritmo Raft implementado, com sucesso.

¹Esta mensagem foi escolhida de forma arbitrária e única para simplificar e acelerar a exploração dos possíveis estados do sistema, sendo suficiente para verificar todas as propriedades.

Faltas

Para analisar o comportamento do Raft perante falhas bizantinas e comportamentos maliciosos, criámos uma “API de Faltas” que permite injetar código em pontos críticos da execução. Pontos críticos são por exemplo, numa receção de mensagem, numa transição de estado importante (exemplo: líder eleito), etc. Desta forma, conseguimos simular anomalias sem alterar o código do algoritmo Raft diretamente.

O impacto de cada falha foi avaliado com testes unitários (usando `cargo test`) com a integração com o [Stateright](#), permitindo verificar se as propriedades eram violadas sob essas condições, e se a mitigação de cada falha era eficaz.

Nas secções seguintes, detalhamos as faltas específicas investigadas, o seu impacto potencial no sistema, a demonstração da sua ocorrência, as possíveis medidas de mitigação e a demonstração dessas medidas.

Falta A - Falsificação/Adulteração de Mensagens

Identificação da Falta

Um nodo malicioso pode falsificar ou adulterar (via *man-in-the-middle*) mensagens de outros nodos.

Impacto

O impacto é total, um nodo malicioso pode tomar controlo total do cluster de nodos. Um nodo malicioso pode:

- fazer com que eleições sejam derrubadas, falsificando mensagens de votação;
- A ADICIONAR MAIS

Demonstração do Impacto

A demonstração de falsificação via Maelstrom é simples, visto que é possível colocar qualquer "src" na mensagem e o Maelstrom não valida a origem da mensagem. Fora do Maelstrom, num sistema com comunicação via IP, também é possível falsificar mensagens, por exemplo, com recurso a *IP Spoofing*.

A FAZER

Medidas de Mitigação

Seria possível mitigar completamente este problema adicionando autenticação às mensagens, por exemplo, através de assinaturas digitais.

Demonstração das Medidas de Mitigação

Devido à natureza do problema, não relacionado com os conteúdos abordados na unidade curricular, não iremos implementar a mitigação.

Falta B - Voto Duplo

Identificação da Falta

Um nodo malicioso pode votar em dois candidatos diferentes numa eleição.

Impacto

Permite que dois nodos sejam eleitos como líder, o que quebra a propriedade de **Election Safety** do algoritmo.

Demonstração do Impacto

A FAZER

Medidas de Mitigação

É possível resolver este problema com uma mensagem adicional, por exemplo, *ElectedBy*, que é enviada num fim de eleição, por todos os nodos que foram eleitos para todos os outros nodos, com a lista de nodos que votaram nele. Caso seja detetado um nodo que votou em dois candidatos diferentes, outros nodos podem ignorar o seu voto futuramente (*black-listed*).

Daqui surge um novo problema, que é, um nodo bizantino pode dizer que recebeu votos de quem não votou nele, o que faria que esse outro nodo fosse *black-listed* indevidamente. Uma solução possível a este problema é recorrer a assinaturas digitais, que permitiriam identificar a autenticidade do voto.

Demonstração das Medidas de Mitigação

A FAZER (sem criptografia)

Falta C - Negação de Serviço por Spam de Eleições

Identificação da Falta

Cada *follower* tem uma *timer* para dar *timeout* e começar uma eleição, quando não recebe atualizações de um líder. Um nodo malicioso pode simplesmente ignorar esse *timer* e começar sempre uma nova eleição. A cada momento desses, o nodo malicioso, incrementa o seu *term* e tenta eleger-se como líder. Como o seu *term* é maior que o do líder, todos os nodos (incluindo o líder) votam nele, e ele passa a ser o líder.

Este nodo malicioso pode continuar a fazer isto indefinidamente, sempre que um líder é eleito.

Impacto

Como não há um líder estável, a propriedade de *liveness* não é respeitada, isto é, o sistema não consegue fazer progresso, visto que não há tempo suficiente para replicar as entradas no log.

Demonstração do Impacto

A FAZER

Falar que o stateright encontrou uma possibilidade de ainda haver lider na implementação do spam.

Medidas de Mitigação

Para este problema, não há uma solução concreta, e teremos que recorrer a soluções heurísticas.

Uma solução possível é ignorar começos de eleições demasiado cedo, por exemplo, se um *term* começou há menos de 1 minuto, ignorar o começo dele. No entanto, esta solução pode fazer com que o sistema volte demasiado tempo a voltar ao normal caso o líder realmente morra nesse espaço de tempo.

Outra solução possível é limitar o número de eleições que um nodo pode iniciar num dado espaço de tempo, por exemplo, 1 eleição por hora.

Demonstração das Medidas de Mitigação

A FAZER

Falta D - Bifurcação de log

Identificação da Falta

Um nodo malicioso pode enviar, no mesmo índice e termo, duas versões diferentes do *log* para *subsets* distintos de nodos. Com isto, diferentes *quorum's* de nodos recebem versões diferentes do *log*, o que faz com que o *log* se bifurque.

Impacto

Se dois conjuntos de nodos aplicarem entradas diferentes no mesmo índice, a propriedade de **Log Safety** é violada: leituras ou escritas podem comportar-se de forma inconsistente em caso de falhas do líder.

Demonstração do Impacto

A FAZER

Medidas de Mitigação

Uma possível solução para esta falta, seria aquando a receção de um *LogRequest* contendo novas entradas do *log*, enviar para todos os outros nodos, uma mensagem adicional que contivesse essas novas entradas. Desta forma, os outros nodos veriam que o *log* não é consistente e começariam uma nova eleição, bloqueando o nodo malicioso.

Para evitar grande tráfego de mensagens, podemos aliviar o envio dessa mensagem adicional para apenas alguns nodos e periodicamente, e não a cada *LogRequest*, como é no protocolo Gossip.

Demonstração das Medidas de Mitigação

A FAZER

Falta E - Commit de log sem ser aceite pela maioria

Identificação da Falta

Um nodo líder malicioso pode incrementar o *commit_index* sem ter recebido confirmação da maioria dos nodos.

Impacto

Os *logs* podem não estar atualizados numa maioria dos nodos, o que faria com que caso o líder falhasse, dados seriam perdidos. Quebra o princípio de *safety*.

Demonstração do Impacto

A FAZER

Medidas de Mitigação

Este problema poderia ser resolvido, enviando uma assinatura digital como forma de autenticidade e autenticação em cada *LogResponse*. Assim, o líder era obrigado a guardar essas assinaturas, para as propagar a seguir. Os outros nodos só aceitariam *LogRequest's* que incrementem o *commit_index*, se nela conter assinaturas digitais válidas de um quorum de nodos.

Demonstração das Medidas de Mitigação

Da mesma forma que na solução da Secção Falta A, devido à natureza do problema, que não está relacionada com os conteúdos abordados na unidade curricular, não iremos implementar a mitigação.